SFTP & SSH guide

Here is the fully updated note. Sections 1-4 and 6 remain exactly as they were, but **Section 5 (SFTP)** has been significantly expanded to explain the "dual-environment" mental model, how to navigate both file systems properly, and how to transfer entire directories.

---

tags:

- networking
- ssh
- ubuntu
- fedora
- linux

aliases:

- SSH Setup Guide
- OpenSSH Server Configuration
- SFTP Guide

SSH & SFTP Server Configuration Guide

Secure Shell (SSH) is a cryptographic network protocol used to operate network services securely over an unsecured network. It is the standard way to remotely log into and manage Linux servers.

1. Checking & Installing the SSH Server

Before configuring anything, we need to ensure the SSH daemon (`sshd`) is actually installed on your machine.

Check if SSH is already installed:

Bash

```
which sshd
```

🧠 **Why are we doing this?**

This command looks for the SSH Daemon binary in your system path. If it returns a path (like `/usr/sbin/sshd`), the server software is installed. If it returns nothing, you need to install it.

Installation

Ubuntu / Debian:

Bash

```
sudo apt update
sudo apt install openssh-server -y
```

Fedora / RHEL:

Bash

```
sudo dnf install openssh-server -y
```

🧠 **Why are we doing this?**

Most desktop Linux versions come with the SSH *client* installed (so you can connect to others), but not the SSH *server* (so others can connect to you). This package installs the background service that listens for incoming remote connections.

Uninstallation (If you need to start fresh)

Ubuntu: `sudo apt purge openssh-server`

Fedora: `sudo dnf remove openssh-server`

---

2. Managing the SSH Service

Once installed, you control SSH using `systemctl`.

(Note: On Ubuntu, the service is called `ssh`. On Fedora, it is called `sshd`.)

Check Status

Bash

```
sudo systemctl status ssh
```

Enable on Boot

Bash

```
sudo systemctl enable ssh
```

🧠 **Why enable it?**

If you reboot your computer, you want SSH to start automatically so you can immediately connect to it remotely. `enable` creates the necessary startup hooks, while `start` only turns it on for the current session.

Disable (For Security)

Bash

```
sudo systemctl disable ssh
```

🧠 **Why disable it?**

If you are taking your laptop to a public cafe network and won't be using remote access, disabling SSH ensures no one on the public Wi-Fi can even attempt to log into your machine.

---

3. Configuration & Security (Changing the Port)

By default, SSH listens on **Port 22**. Because this is universal, hackers constantly run automated bots to attack Port 22. Changing the port is a great first step for security.

Edit the Configuration File

Bash

```
sudo nano /etc/ssh/sshd_config
```

🧠 **Why this specific file?**

`/etc/ssh/` contains all SSH settings.

- `ssh_config` (no 'd') is for your *client* (when you connect to others).
- `sshd_config` (with the 'd' for daemon) is for your *server* (when others connect to you).

Make the Change

Inside the file, look for a line that says `#Port 22`.

1. **Uncomment it** (remove the `#`).
2. **Change the number** to something custom (e.g., `Port 2222`).

Apply the Changes

To make the new port active, you must restart the service:

Bash

```
sudo systemctl restart ssh
```

(Alternatively, you can run `sudo reboot` to restart the entire machine, but restarting just the service is much faster and standard practice for Linux admins).

---

4. Connecting via SSH (The Client)

Now that the server is running, here is how you connect to it from *another* computer.

Standard Login

Bash

```
ssh username@192.168.1.10
```

Login with a Custom Port

If you changed the port in Step 3, the standard command will fail. You must specify the new port using the `-p` flag:

Bash

```
ssh -p 2222 username@192.168.1.10
```

🧠 **Why no `sudo`?**

You do not need (and shouldn't use) `sudo` to run the `ssh` client command. SSH operates perfectly under your normal user privileges. You only need `sudo` when modifying the *server* configurations.

---

5. Secure File Transfer (SFTP)

SFTP (SSH File Transfer Protocol) is a subsystem of SSH. If SSH is working, SFTP is automatically working. While standard SSH gives you a remote terminal to *run commands*, SFTP gives you a secure session exclusively designed to *move files* back and forth.

The Mental Model: Two Computers at Once

When you log into SFTP, your terminal is technically connected to **both** your Local PC and the Remote Server simultaneously. Because of this, you need specific commands to tell the terminal which machine's folders you want to interact with.

Connecting via SFTP

(Note: SFTP uses a capital `-P` for the port flag, unlike SSH which uses a lowercase `-p`).

Bash

```
sftp -P 2222 username@192.168.1.10
```

Once connected, your prompt will change from `$` to `sftp>`.

Navigating the Folders (Remote vs. Local)

Before transferring files, you must navigate to the correct folder on *both* machines.

- **Standard Commands (Remote Server):** `pwd`, `cd`, `ls`
- **Local Commands (Your PC):** Add an `l` (for local) to the front: `lpwd`, `lcd`, `lls`

🧠 **Why the difference?**

If you type `ls`, the terminal assumes you want to see the files on the *Remote Server*. If you want to see the files in the folder you are currently sitting in on your *Local PC*, you have to explicitly type `lls` (local list). You use `cd` to change the server's directory, and `lcd` to change your own computer's directory.

Transferring Files

Once you have navigated to the right folders on both sides, you can move the data.

1. Downloading (Remote Local)

To pull a file from the server down to your PC:

Bash

```
get document.pdf
```

To pull an entire directory (folder), use the recursive flag `-r`:

Bash

```
get -r /var/www/sajib
```

2. Uploading (Local Remote)

To push a file from your PC up to the server:

Bash

```
put mycode.py
```

To push an entire directory:

Bash

```
put -r local_folder/
```

🧠 **Why use SFTP instead of standard FTP?**

Standard FTP sends all files and passwords in plain text, meaning anyone on your local network could use a packet sniffer to steal them. SFTP encrypts the entire transfer using your SSH keys, keeping your code and passwords completely safe.

Exiting the Session

When you are done moving files, close the connection to return to your normal terminal.

Bash

```
exit
```

---

6. Troubleshooting & Maintenance

Checking if the Port is Open

If you can't connect, verify that your server is actually listening on the port.

Bash

```
sudo lsof -i -n
```

Or, use the standard socket stats command:

Bash

```
ss -tlpn | grep LISTEN
```

🧠 **Why do this?**

This confirms that the `sshd` process is running and tells you exactly which port number it is currently bound to. If you don't see `sshd` here, the service crashed or wasn't started.

Fixing the "Host Identification Changed" Error

If you reinstall the OS on your server but keep the same IP address, your client PC will panic and refuse to connect, throwing a giant `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!` error.

The Fix:

Delete the old, mismatched security fingerprint from your local computer.

Bash

```
rm ~/.ssh/known_hosts
```

🧠 **Why does this happen?**

The first time you SSH into a server, your PC saves the server's unique "fingerprint" in the `.ssh/known_hosts` file. If the server's fingerprint suddenly changes (because you reinstalled the OS or swapped the machine), SSH assumes a hacker is trying to intercept your connection and blocks you to keep you safe. Deleting the file forces SSH to "re-learn" the new fingerprint.Here is the fully updated note. Sections 1-4 and 6 remain exactly as they were, but **Section 5 (SFTP)** has been significantly expanded to explain the "dual-environment" mental model, how to navigate both file systems properly, and how to transfer entire directories.

---

tags:

- networking
- ssh
- ubuntu
- fedora
- linux

aliases:

- SSH Setup Guide
- OpenSSH Server Configuration

- SFTP Guide

SSH & SFTP Server Configuration Guide

Secure Shell (SSH) is a cryptographic network protocol used to operate network services securely over an unsecured network. It is the standard way to remotely log into and manage Linux servers.

1. Checking & Installing the SSH Server

Before configuring anything, we need to ensure the SSH daemon (`sshd`) is actually installed on your machine.

Check if SSH is already installed:

Bash

```
which sshd
```

🧠 **Why are we doing this?**

This command looks for the SSH Daemon binary in your system path. If it returns a path (like `/usr/sbin/sshd`), the server software is installed. If it returns nothing, you need to install it.

Installation

Ubuntu / Debian:

Bash

```
sudo apt update
sudo apt install openssh-server -y
```

Fedora / RHEL:

Bash

```
sudo dnf install openssh-server -y
```

🧠 **Why are we doing this?**

Most desktop Linux versions come with the SSH *client* installed (so you can connect to others), but not the SSH *server* (so others can connect to you). This package installs the background service that listens for incoming remote connections.

Uninstallation (If you need to start fresh)

Ubuntu: `sudo apt purge openssh-server`

Fedora: `sudo dnf remove openssh-server`

---

2. Managing the SSH Service

Once installed, you control SSH using `systemctl`.

(Note: On Ubuntu, the service is called `ssh`. On Fedora, it is called `sshd`.)

Check Status

Bash

```
sudo systemctl status ssh
```

Enable on Boot

Bash

```
sudo systemctl enable ssh
```

🧠 **Why enable it?**

If you reboot your computer, you want SSH to start automatically so you can immediately connect to it remotely. `enable` creates the necessary startup hooks, while `start` only turns it on for the current session.

Disable (For Security)

Bash

```
sudo systemctl disable ssh
```

🧠 **Why disable it?**

If you are taking your laptop to a public cafe network and won't be using remote access, disabling SSH ensures no one on the public Wi-Fi can even attempt to log into your machine.

---

3. Configuration & Security (Changing the Port)

By default, SSH listens on **Port 22**. Because this is universal, hackers constantly run automated bots to attack Port 22. Changing the port is a great first step for security.

Edit the Configuration File

Bash

```
sudo nano /etc/ssh/sshd_config
```

🧠 **Why this specific file?**

`/etc/ssh/` contains all SSH settings.

- `ssh_config` (no 'd') is for your *client* (when you connect to others).
- `sshd_config` (with the 'd' for daemon) is for your *server* (when others connect to you).

Make the Change

Inside the file, look for a line that says `#Port 22`.

1. **Uncomment it** (remove the `#`).
2. **Change the number** to something custom (e.g., `Port 2222`).

Apply the Changes

To make the new port active, you must restart the service:

Bash

```
sudo systemctl restart ssh
```

(Alternatively, you can run `sudo reboot` to restart the entire machine, but restarting just the service is much faster and standard practice for Linux admins).

---

4. Connecting via SSH (The Client)

Now that the server is running, here is how you connect to it from *another* computer.

Standard Login

Bash

```
ssh username@192.168.1.10
```

Login with a Custom Port

If you changed the port in Step 3, the standard command will fail. You must specify the new port using the `-p` flag:

Bash

```
ssh -p 2222 username@192.168.1.10
```

🧠 **Why no `sudo`?**

You do not need (and shouldn't use) `sudo` to run the `ssh` client command. SSH operates perfectly under your normal user privileges. You only need `sudo` when modifying the *server* configurations.

---

5. Secure File Transfer (SFTP)

SFTP (SSH File Transfer Protocol) is a subsystem of SSH. If SSH is working, SFTP is automatically working. While standard SSH gives you a remote terminal to *run commands*, SFTP gives you a secure session exclusively designed to *move files* back and forth.

The Mental Model: Two Computers at Once

When you log into SFTP, your terminal is technically connected to **both** your Local PC and the Remote Server simultaneously. Because of this, you need specific commands to tell the terminal which machine's folders you want to interact with.

Connecting via SFTP

(Note: SFTP uses a capital `-P` for the port flag, unlike SSH which uses a lowercase `-p`).

Bash

```
sftp -P 2222 username@192.168.1.10
```

Once connected, your prompt will change from `$` to `sftp>`.

Navigating the Folders (Remote vs. Local)

Before transferring files, you must navigate to the correct folder on *both* machines.

- **Standard Commands (Remote Server):** `pwd`, `cd`, `ls`
- **Local Commands (Your PC):** Add an `l` (for local) to the front: `lpwd`, `lcd`, `lls`

🧠 **Why the difference?**

If you type `ls`, the terminal assumes you want to see the files on the *Remote Server*. If you want to see the files in the folder you are currently sitting in on your *Local PC*, you have to explicitly type `lls` (local list). You use `cd` to change the server's directory, and `lcd` to change your own computer's directory.

Transferring Files

Once you have navigated to the right folders on both sides, you can move the data.

1. Downloading (Remote Local)

To pull a file from the server down to your PC:

Bash

```
get document.pdf
```

To pull an entire directory (folder), use the recursive flag `-r`:

Bash

```
get -r /var/www/sajib
```

2. Uploading (Local Remote)

To push a file from your PC up to the server:

Bash

```
put mycode.py
```

To push an entire directory:

Bash

```
put -r local_folder/
```

🧠 **Why use SFTP instead of standard FTP?**

Standard FTP sends all files and passwords in plain text, meaning anyone on your local network could use a packet sniffer to steal them. SFTP encrypts the entire transfer using your SSH keys, keeping your code and passwords completely safe.

Exiting the Session

When you are done moving files, close the connection to return to your normal terminal.

Bash

```
exit
```

---

6. Troubleshooting & Maintenance

Checking if the Port is Open

If you can't connect, verify that your server is actually listening on the port.

Bash

```
sudo lsof -i -n
```

Or, use the standard socket stats command:

Bash

```
ss -tlpn | grep LISTEN
```

🧠 **Why do this?**

This confirms that the `sshd` process is running and tells you exactly which port number it is currently bound to. If you don't see `sshd` here, the service crashed or wasn't started.

Fixing the "Host Identification Changed" Error

If you reinstall the OS on your server but keep the same IP address, your client PC will panic and refuse to connect, throwing a giant `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!` error.

The Fix:

Delete the old, mismatched security fingerprint from your local computer.

Bash

```
rm ~/.ssh/known_hosts
```

🧠 **Why does this happen?**

The first time you SSH into a server, your PC saves the server's unique "fingerprint" in the `.ssh/known_hosts` file. If the server's fingerprint suddenly changes (because you reinstalled the OS or swapped the machine), SSH assumes a hacker is trying to intercept your connection and blocks you to keep you safe. Deleting the file forces SSH to "re-learn" the new fingerprint.